EXTDRIFT · CSD493 PROJECT-1 · DATA & MACHINE-LEARNING TRACK

Dataset Generation and Model Training

A step-by-step working guide — synthetic data, real data, and training the model

For: Suhani Deepak Agrawat (data & ML lead)

With: Bind Pratap Singh (infrastructure)

Advisor: Dr. Sweta Mishra · **Date:** September 2026

What this guide gives you. Everything needed to build the dataset and train the model, from a clean checkout to a saved, evaluated classifier — every command, what it should print, what each part means, and how to extend it. Start with the synthetic data (Part A, works today with no downloads), run the training (Part C) to see the full workflow, and collect real data (Part B) in parallel because it accrues over weeks. Heavy runs should use the MacBook.

Contents

0 Setup — one-time

A The synthetic dataset (start here)

B The real dataset (collect in parallel)

C Train and evaluate the model

D Evaluation hygiene & troubleshooting

E Your milestones

> **The whole workflow on one line.** labelled update pairs → `engine.batch` builds a table of behavioural deltas (`data/dataset/deltas.csv`) → `engine.ml.bakeoff` trains and compares models and saves the winner → `engine.ml.anomaly` checks unseen threats. Parts A/B build the pairs; Part C runs the training.

0 Setup — one-time

0.1 Get the code and a Python environment

Python 3.11+ and the repository. Create an isolated environment so libraries don't clash:

```
# from the repo root
python -m venv .venv
# Windows: .venv\Scripts\activate   # Mac: source .venv/bin/activate
```

0.2 Install the libraries

The analysis half needs nothing extra. The ML half needs these (numpy, pandas, xgboost are usually already present; the two that must be added are `scikit-learn` and `joblib`):

```
pip install scikit-learn joblib numpy pandas xgboost
# only if you will also capture real extensions in a browser:
pip install playwright && playwright install chromium
```

0.3 Confirm everything runs

```
python -m unittest discover -s tests
```

> EXPECTED (ROUGHLY)
>
> `Ran 80 tests ... OK`

If the ML tests are skipped, scikit-learn is not installed in this environment — redo step 0.2.

A The synthetic dataset — start here

Confirmed malicious *update pairs* are rare in the wild, and no public labelled set exists, so we generate our own. This is the standard, honest "synthetic-train / real-validate" design.

A.1 Understand what a "pair" is (read once)

- A **pair** is two versions of the *same* synthetic extension: `v1` (trusted) and `v2` (the update). Each is a synthetic *behavioural record* (a "trace").
- The **label lives in the difference**, not the raw counts. Example: a password-manager persona reads password fields in *both* versions, so that alone is not suspicious — only a *new* read-then-upload in v2 is. This is deliberate, so the task is not trivially separable.
- Each extension has a **persona** (reader, shopper, contacts-scraper, password-manager, weather, translate) and each malicious pair injects one of six **payload families**: `cookie_theft`, `credential_theft`, `sw_beacon`, `ad_affiliate`, `keylogger`, `history_exfil` — half of them in an *obfuscated* variant (identical behaviour, hidden code).

A.2 Generate the dataset

```
python -m engine.batch --extensions 60 --updates 3 --seed 1234 --out data/dataset/deltas.csv
```

EXPECTED OUTPUT

```
[dataset] 180 rows · 60 extensions · 27 features
[labels ] {'benign': 103, 'malicious': 77}
[written] data/dataset/deltas.csv
```

Each row is one update pair reduced to numbers. `--extensions` and `--updates` control the size; `--seed` makes it reproducible.

A.3 What the columns mean

Group	Columns	Meaning
Identity	`pair_id, ext_id, name, persona, collected_at`	Which pair; `ext_id` and `collected_at` are used to split the data without cheating (Part D)
Label	`label, family, obfuscated, source`	benign / malicious; which payload; whether the code was hidden
Rule score	`rule_score`	The transparent heuristic score, so the rules are a baseline in training
Behavioural delta (24)	`net_*, dom_*, cookie_*, api_*, perm_*, exfil_*`	The Δ = v2 − v1 features: new hosts, uploads, cookie reads, injections, read-then-upload flows, permission changes, service-worker traffic, …
Static diff (3)	`stat_added_highrisk_perms, stat_escalated_broad_host, stat_added_hosts`	Cheap signals from the manifest, available before the extension even runs

A.4 Inspect it — sanity check

```
python -c "import pandas as pd; d=pd.read_csv('data/dataset/deltas.csv'); \
print(d['label'].value_counts().to_dict()); \
print(d[d.label=='malicious']['family'].value_counts().to_dict())"
```

Look for: both labels present; several extensions; malicious rows showing non-zero exfiltration features; some benign rows with a new host or a small upload (these are the *hard* benign cases that create realistic false-positive pressure — they are meant to be there).

A.5 Extend it — add a payload family (when needed)

Open `engine/synth/generate.py`. Add a name to `PAYLOAD_FAMILIES`, then add a branch to `_apply_family(...)` that appends the family's runtime effects to the v2 trace — for example a new sensitive read plus a POST to a fresh collector host. Follow the existing branches as templates. Regenerate with A.2. Nothing else needs to change; the feature extractor and training pick it up automatically.

B The real dataset — collect in parallel

Real data is what proves the **false-positive** claim (that we do not flag ordinary updates). Real benign update pairs are easy to obtain; start now because some take weeks to appear.

B.1 Route 1 — open-source release pairs (best, do first)

1. Pick an open-source extension on GitHub (e.g. uBlock Origin, Dark Reader, Stylus, Vimium).
2. Download two consecutive tagged releases and build/unzip each into a folder with a `manifest.json` (call them `v1/` and `v2/`).
3. Capture both versions through the real pipeline; this writes the two behavioural records:

```
python -m engine.cli run-pair path/to/v1 path/to/v2 \
    --out-dir data/dataset/captured/darkreader_4.9.6_to_4.9.7
```

Then add a small label file next to the traces so the dataset builder knows it is a real benign update:

```
# data/dataset/captured/<name>/label.json
{"label":"benign","family":"","obfuscated":false,"source":"github","collected_at":"2026-09-15"}
```

A real update that you have independently confirmed is malicious would be labelled `"malicious"`; if you are unsure, use `"unknown"` and keep it out of training.

B.2 Route 2 — historical archives (for closed-source, popular ones)

Sites such as `crx4chrome.com` host old `.crx` files, and `chrome-stats.com` shows version timelines. Download two versions, then unpack and capture exactly as in B.1 (`python -m engine.cli unpack file.crx out/` first, then `run-pair`). Note in the write-up that these archives are unofficial and integrity cannot be fully verified.

B.3 Route 3 — weekly self-snapshots (real auto-updates)

Chrome keeps installed extensions unpacked on disk. Copy the current version folder now; when Chrome auto-updates, copy the new folder before it is removed — that gives a genuine real update pair.

```
# Windows
C:\Users\<you>\AppData\Local\Google\Chrome\User Data\Default\Extensions\<id>\<version>\
# Mac
~/Library/Application Support/Google/Chrome/Default/Extensions/<id>/<version>/
```

Pick ~15 extensions to watch; snapshot weekly. Bind will script this, but the choice of which extensions to watch is yours.

B.4 Fold the real pairs into the dataset

```
python -m engine.batch --extensions 60 --updates 3 \
    --captured data/dataset/captured --out data/dataset/deltas.csv
```

Any folder under `data/dataset/captured/` that contains `v1.trace.json` and `v2.trace.json` is added as an extra row, using its `label.json`. The synthetic rows and the real rows now live in one table.

> **Safety, every time.** Only *synthetic* samples are ever labelled malicious by us. Real downloaded extensions are treated as untrusted: unpack and run them only inside the sandbox, against the fake test pages, never against real accounts, and never redistribute them. Review each synthetic payload before it enters `data/`.

C Train and evaluate the model

C.1 Run the model bake-off

This trains and fairly compares five approaches on the identical features and saves the winner.

```
python -m engine.ml.bakeoff data/dataset/deltas.csv
```

EXPECTED OUTPUT (SYNTHETIC DATA)

```
model       PR-AUC  ROC-AUC   prec  recall     F1  FPR@rec90
------------------------------------------------------------
rf           0.999    0.999  1.000   0.948  0.973      0.000  <= winner
svm          0.998    0.998  0.974   0.961  0.967      0.000
logreg       0.997    0.997  0.960   0.935  0.947      0.000
xgboost      0.978    0.970  0.987   0.974  0.980      0.010
rules        0.868    0.884  0.632   0.870  0.732      0.660

Winner: RF (selected by PR-AUC, tie-broken by FPR at 90% recall)
Does ML earn its place? YES — the learned model beats the rules.
```

The fitted winner and its metrics are written to `engine/ml/models/`.

C.2 How to read the table

- **PR-AUC** (precision–recall area) — the main score when malicious pairs are the minority. Prefer it over accuracy, which is misleading on imbalanced data.
- **FPR at 90% recall** — the false-positive rate you pay to catch 90% of attacks. This is the number that decides whether a detector is usable. *Lower is better.*
- **The rules row is the control.** If a learned model cannot beat the rules here, that is a legitimate finding to report — not a failure. On the current data it beats them clearly (rules flag 66% of benign updates at 90% recall; the model flags ~0%).
- **Why "split by extension"** — all versions of one extension are kept together in one fold, so the model is judged on extensions it has never seen, not on ones it memorised.

To see the features the winner relies on, the same command prints a ranked importance list (`net_upload_bytes`, `dom_injections`, … on the current data).

C.3 Test against unseen threats (the anomaly layer)

```
python -m engine.ml.anomaly data/dataset/deltas.csv
```

EXPECTED OUTPUT

```
held-out family      detected  detect-rate  benign-FPR  ROC-AUC
cookie_theft            14/14         1.00        0.06     1.00
sw_beacon               18/18         1.00        0.06     1.00
...    (each family was NEVER seen during training)
```

This trains only on benign updates, then hides an entire malicious family and checks whether it is still flagged as anomalous. A high detection rate means the system reacts to genuinely novel malicious *behaviour* — which is the honest version of "detecting the unknown". It does **not** mean we find unknown code vulnerabilities; do not claim that.

C.4 Use the saved model on any new pair

```
python -c "
import joblib, numpy as np
from engine.batch import rows_from_captured
w = joblib.load('engine/ml/models/winner.joblib')
for r in rows_from_captured('data/dataset/captured'):
    x = np.array([[float(r.get(f,0) or 0) for f in w['features']]])
    p = w['model'].predict_proba(x)[0,1]
    print(r['pair_id'], round(p,3), 'MALICIOUS' if p>=0.5 else 'BENIGN')"
```

EXAMPLE (OUR LIVE BROWSER CAPTURES)

```
readerlite_benign     0.413  BENIGN
readerlite_malicious  0.840  MALICIOUS
```

This is the "real-pipeline validation": a model trained on synthetic data judging genuinely captured behaviour correctly.

D Evaluation hygiene & troubleshooting

Keep the evaluation honest — checklist

- ✓ **Split by extension** (already done): never let two versions of the same extension sit in both train and test.
- ✓ **Split by time** for the drift check: train on older pairs, test on newer ones.
- ✓ **Report PR-AUC and FPR**, not accuracy alone — the classes are imbalanced.
- ✓ **Remove identifiers**: the model must key on *behavioural change*, not on which library or extension it is looking at.
- ✓ **Hold out whole families** to test the anomaly layer's reaction to the unseen.

Common issues

Symptom	Cause & fix
`ModuleNotFoundError: sklearn`	scikit-learn not installed — redo step 0.2.

Symptom	Cause & fix
Cross-validation error about folds/groups	Too few extensions or too few of one class for the number of folds — generate more data (raise `--extensions`) or it will automatically reduce folds.
Scores look "too perfect" (≈1.0)	Expected on synthetic data; it is clean. The real test is on real data — collect it (Part B) and re-run.
A learned model does not beat the rules	A valid, honest finding at small scale — report it; more or harder data usually changes it.

E Your milestones (data & ML track)

1. **This week:** run Parts A and C end-to-end (already working); choose ~15 extensions to watch and grab 2–3 GitHub release pairs (Part B.1) to seed the real set.
2. **Weeks 2–4:** grow the synthetic corpus (more families/bases via A.5); accumulate real benign pairs; fold them in (B.4).
3. **Weeks 4–6:** re-run the bake-off on the combined data; report PR-AUC and false-positive rate on the *real* benign pairs; lock the model choice with evidence.
4. **Later:** the write-up's results section — the model comparison, the anomaly result, and the false-positive evaluation.

---

extdrift · CSD493 Project-1 · Data & ML working guide for Suhani Deepak Agrawat. Commands assume the repository root as the working directory. Reported figures are from the current synthetic corpus and live captures; the real-data evaluation is the next milestone.